

# **E-Commerce Order Tracking System - DBMS Project Report**

**PROJECT TITLE: E-Commerce Order Tracking System**

**TEAM DETAILS:**

| <b>NAMES</b>               | <b>SRN</b>           |
|----------------------------|----------------------|
| <b>VIKRAM HALLI</b>        | <b>PES2UG23CS690</b> |
| <b>TUSHAR KSHEERASAGAR</b> | <b>PES2UG23CS658</b> |

## **2. Abstract**

This project presents an E-Commerce Order Tracking System designed to manage customers, products, orders, shipments, returns, and reviews.

The database ensures data consistency using triggers, stored procedures, functions, and relational integrity.

It supports automated stock updates, order total calculations, return processing, sales reporting, and customer/product analytics.

## **3. User Requirement Specification**

### **->Purpose of the project**

The purpose is to provide a reliable backend database system for an e-commerce site that automates inventory control, order accounting and return handling, and supports analytics and reporting. The system ensures data integrity (no negative stock, correct totals), speeds up common administrative tasks (customer order lookup, returns processing), and exposes database functions for business metrics (product revenue, customer LTV, average ratings).

## **->Scope of the project**

This project covers the database layer and server-side logic for order tracking and reporting. It includes table schemas for products, customers, orders, order\_details, shipments, returns and reviews; triggers to enforce and automate inventory and order total updates; stored procedures for common operations (order history, returns processing, sales reports); and functions for analytics (revenues, LTV, ratings). The scope excludes front-end UI design, payment gateway integration, and shipping carrier APIs — although the schema supports storing shipment numbers and delivery dates for integration later.

## **->Detailed description**

A mid-sized online retailer sells multiple products to registered customers. When a customer places an order, the system records the order and its line items in ORDERS and ORDER\_DETAILS. An AFTER INSERT trigger on ORDER\_DETAILS decrements the corresponding PRODUCTS.stock\_quantity and recalculates ORDERS.total\_amount so totals and inventory remain synchronized. Updates to PRODUCTS are protected with a BEFORE UPDATE trigger preventing negative stock values.

If a customer requests a return, an entry goes into RETURNS. A trigger and/or a stored procedure handles status updates: when a return is approved the system restores stock (increments PRODUCTS.stock\_quantity) and updates the return status. Shipments are tracked in SHIPMENTS (tracking number, delivery date) and linked to orders so customers and admins can view shipment status.

For reporting and admin tasks there are stored procedures:

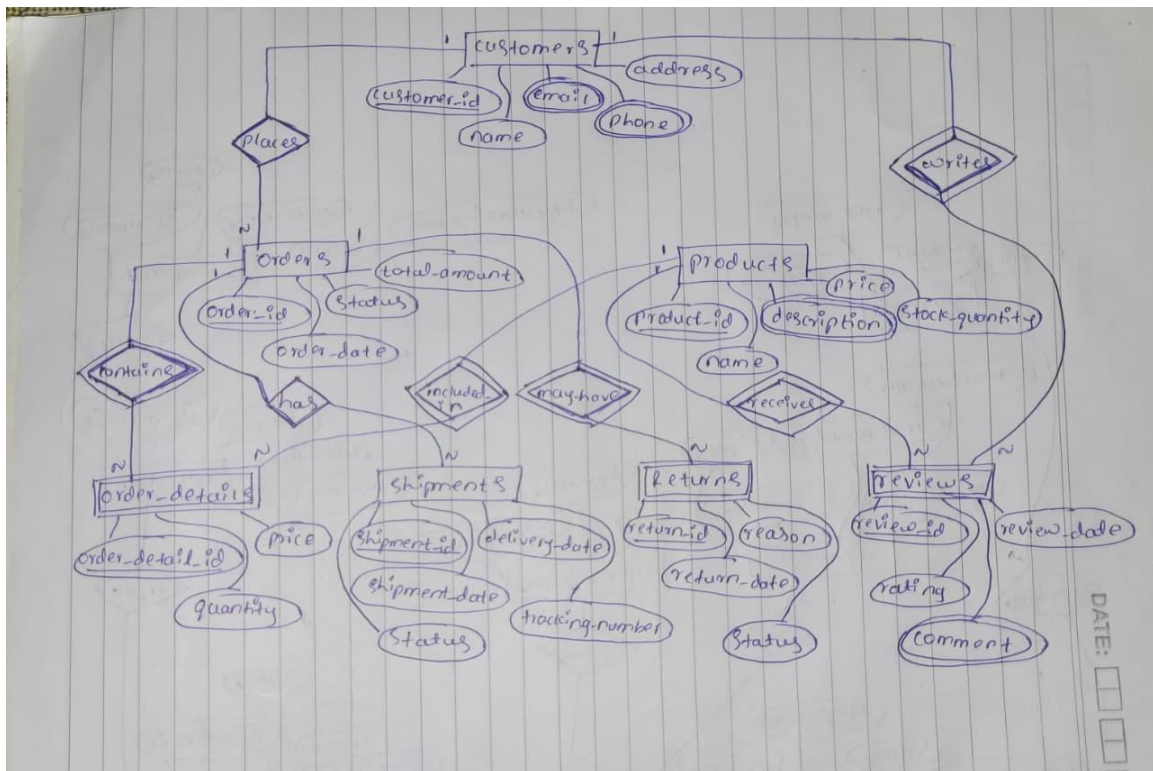
get\_customer\_order\_history(cust\_id) returns a customer's orders with product and shipment details; process\_product\_return(return\_id, new\_status) encapsulates return workflows including stock restoration; generate\_sales\_report(start\_date, end\_date) produces daily sales summaries (orders, unique customers, items sold, revenue, average order value). Functions like calculate\_product\_revenue(prod\_id), calculate\_customer\_ltv(cust\_id) and get\_average\_rating(prod\_id) provide

business metrics for dashboards or exports. (See attached SQL file for full code of triggers, procedures and functions.)

## 4. Software / Tools Used

- MySQL Server
- Python 3.x
- Tkinter GUI Library
- mysql-connector-python
- MySQL Workbench / CLI
- Windows/Linux OS

## 5. ER Diagram



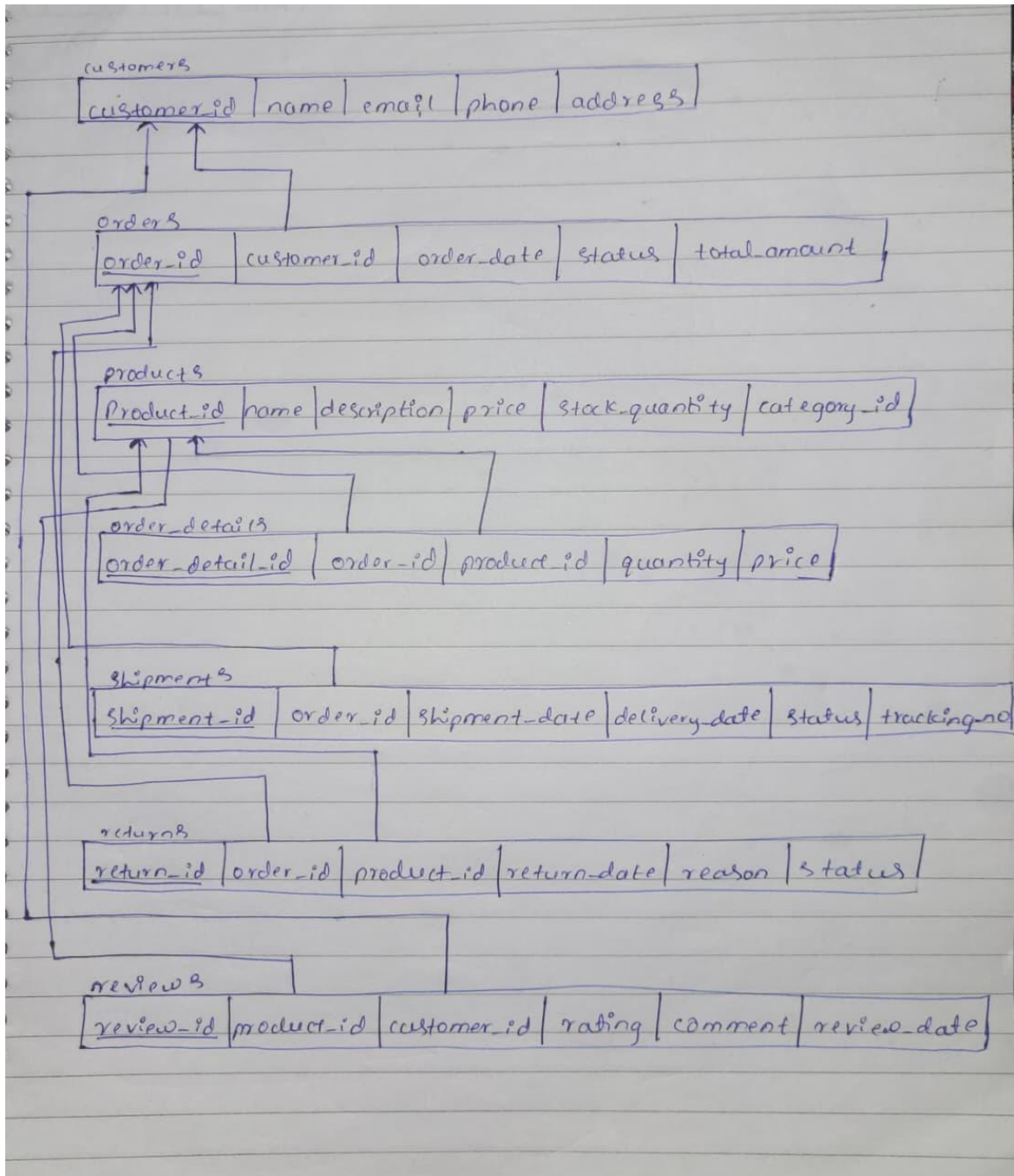
## 6. Relational Schema

CUSTOMERS(customer\_id, name, email, phone, address)

PRODUCTS(product\_id, name, description, price, stock\_quantity)

ORDERS(order\_id, customer\_id, order\_date, status, total\_amount)

ORDER\_DETAILS(order\_detail\_id, order\_id, product\_id, quantity, price)  
 SHIPMENTS(shipment\_id, order\_id, shipment\_date, delivery\_date, status, tracking\_number)  
 RETURNS(return\_id, order\_id, product\_id, return\_date, reason, status)  
 REVIEWS(review\_id, product\_id, customer\_id, rating, comment, review\_date)



## 7. DDL Commands

-- Create the CUSTOMERS table

```
CREATE TABLE CUSTOMERS (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(20) NOT NULL,  
    address VARCHAR(255) NOT NULL  
);
```

-- Create the PRODUCTS table

```
CREATE TABLE PRODUCTS (  
    product_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    description TEXT,  
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
    stock_quantity INT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0));
```

-- Create the ORDERS table

```
CREATE TABLE ORDERS (  
    order_id INT PRIMARY KEY,  
    customer_id INT NOT NULL,  
    order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) NOT NULL DEFAULT 'Pending',  
    total_amount DECIMAL(10,2) NOT NULL CHECK (total_amount >= 0),  
    CONSTRAINT fk_orders_customers FOREIGN KEY (customer_id)  
        REFERENCES CUSTOMERS (customer_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

-- Create the ORDER\_DETAILS table

```
CREATE TABLE ORDER_DETAILS (  
    order_detail_id INT PRIMARY KEY,  
    order_id INT NOT NULL,  
    product_id INT NULL,
```

```
quantity INT NOT NULL CHECK (quantity > 0),
price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
CONSTRAINT fk_orderdetails_orders FOREIGN KEY (order_id)
    REFERENCES ORDERS(order_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
CONSTRAINT fk_orderdetails_products FOREIGN KEY (product_id)
    REFERENCES PRODUCTS(product_id)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);
```

-- Create the SHIPMENTS table

```
CREATE TABLE SHIPMENTS (
    shipment_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    shipment_date DATE NOT NULL,
    delivery_date DATE,
    status VARCHAR(20) NOT NULL DEFAULT 'Shipped',
    tracking_number VARCHAR(50) UNIQUE,
    CONSTRAINT fk_shipments_orders FOREIGN KEY (order_id)
        REFERENCES ORDERS(order_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

-- Create the RETURNS table

```
CREATE TABLE RETURNS (
    return_id INT PRIMARY KEY,
    order_id INT NULL,
    product_id INT NULL,
    return_date DATE NOT NULL,
    reason VARCHAR(255) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Processing',
    CONSTRAINT fk_returns_orders FOREIGN KEY (order_id)
        REFERENCES ORDERS(order_id)
        ON DELETE CASCADE
        ON UPDATE SET NULL,
```

```
CONSTRAINT fk_returns_products FOREIGN KEY (product_id)
REFERENCES PRODUCTS(product_id)
ON DELETE SET NULL
ON UPDATE SET NULL
);
```

-- Create the REVIEWS table

```
CREATE TABLE REVIEWS (
  review_id INT PRIMARY KEY,
  product_id INT NULL,
  customer_id INT NOT NULL,
  rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment TEXT,
  review_date DATE NOT NULL,
  CONSTRAINT fk_reviews_products FOREIGN KEY (product_id)
  REFERENCES PRODUCTS(product_id)
  ON DELETE CASCADE
  ON UPDATE CASCADE,
  CONSTRAINT fk_reviews_customers FOREIGN KEY (customer_id)
  REFERENCES CUSTOMERS(customer_id)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```

-- CUSTOMERS

INSERT INTO CUSTOMERS VALUES

```
(1, 'Amit Sharma', 'amit.sharma@email.com', '9876543210', 'Bangalore, India'),
(2, 'Pooja Singh', 'pooja.singh@email.com', '9876501234', 'Delhi, India'),
(3, 'Rahul Kumar', 'rahul.kumar@email.com', '9812345678', 'Mumbai, India'),
(4, 'Sneha Patel', 'sneha.patel@email.com', '9123456789', 'Ahmedabad, India'),
(5, 'Vikas Roy', 'vikas.roy@email.com', '9001122334', 'Kolkata, India');
```

-- PRODUCTS (Assuming category\_id as 1 for all)

INSERT INTO PRODUCTS VALUES

```
(1, 'Wireless Mouse', 'Ergonomic Wireless Mouse', 550.00, 80),
(2, 'Mechanical Keyboard', 'RGB Backlit Mechanical Keyboard', 2500.00, 50),
(3, '16GB Pen Drive', 'USB 3.1 High Speed Pen Drive', 700.00, 120),
```

```
(4, 'Laptop Stand', 'Adjustable Laptop Stand', 900.00, 40),  
(5, 'Bluetooth Speaker', 'Portable Bluetooth Speaker', 1200.00, 60);
```

```
-- ORDERS
```

```
INSERT INTO ORDERS VALUES
```

```
(1, 1, '2025-08-01', 'Shipped', 3200.00),  
(2, 2, '2025-08-02', 'Delivered', 1450.00),  
(3, 3, '2025-08-03', 'Processing', 900.00),  
(4, 4, '2025-08-04', 'Cancelled', 2500.00),  
(5, 5, '2025-08-05', 'Shipped', 1950.00);
```

```
-- ORDER_DETAILS
```

```
INSERT INTO ORDER_DETAILS VALUES
```

```
(1, 1, 2, 1, 2500.00),  
(2, 1, 5, 1, 1200.00),  
(3, 2, 1, 1, 550.00),  
(4, 2, 3, 2, 700.00),  
(5, 3, 4, 1, 900.00);
```

```
-- SHIPMENTS
```

```
INSERT INTO SHIPMENTS VALUES
```

```
(1, 1, '2025-08-02', '2025-08-05', 'Delivered', 'TRK001'),  
(2, 2, '2025-08-03', '2025-08-06', 'Delivered', 'TRK002'),  
(3, 3, '2025-08-04', NULL, 'Shipped', 'TRK003'),  
(4, 4, '2025-08-06', NULL, 'Cancelled', 'TRK004'),  
(5, 5, '2025-08-07', NULL, 'Shipped', 'TRK005');
```

```
-- RETURNS
```

```
INSERT INTO RETURNS VALUES
```

```
(1, 2, 3, '2025-08-09', 'Defective item', 'Processed'),  
(2, 1, 5, '2025-08-10', 'Not as described', 'Processing'),  
(3, 1, 2, '2025-08-10', 'Wrong item sent', 'Processing'),  
(4, 2, 1, '2025-08-11', 'Damaged in transit', 'Processed'),  
(5, 3, 4, '2025-08-12', 'Changed mind', 'Pending');
```



-- REVIEWS

INSERT INTO REVIEWS VALUES

```
(1, 1, 1, 4, 'Works well, satisfied!', '2025-08-06'),  
(2, 3, 2, 5, 'Very fast data transfer.', '2025-08-07'),  
(3, 5, 1, 2, 'Poor sound quality.', '2025-08-08'),  
(4, 4, 5, 5, 'Sturdy and versatile stand!', '2025-08-09'),  
(5, 2, 3, 3, 'Good keyboard, average keys.', '2025-08-10');
```

## 8. CRUD Operation Screenshots

### CREATE

CreateRead/RefreshUpdateDelete

Search:

|   | customer_id | name        | email                 | phone      | address          |
|---|-------------|-------------|-----------------------|------------|------------------|
| 1 |             | Amit Sharma | amit.sharma@email.com | 9876543210 | Bangalore, India |
| 2 |             | Pooja Singh | pooja.singh@email.com | 9876501234 | Delhi, India     |
| 3 |             | Rahul Kumar | rahul.kumar@email.com | 9812345678 | Mumbai, India    |
| 4 |             | Sneha Patel | sneha.patel@email.com | 9123456789 | Ahmedabad, India |
| 5 |             | Vikas Roy   | vikas.roy@email.com   | 9001122334 | Kolkata, India   |
| 7 |             | tushar      | tushar@gmail.com      | 234567890  | banglore         |

Create New Customers Record

### Enter Customers Details

customer\_id:

8

name:

suhas

email:

suhas@gmail.com

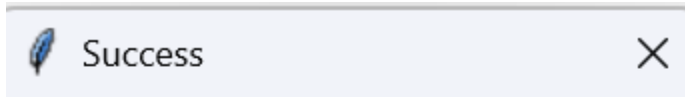
phone:

9876543212

address:

banglore

Save



Record created successfully!

OK

|   | customer_id | name        | email                 | phone      | address          |
|---|-------------|-------------|-----------------------|------------|------------------|
| 1 |             | Amit Sharma | amit.sharma@email.com | 9876543210 | Bangalore, India |
| 2 |             | Pooja Singh | pooja.singh@email.com | 9876501234 | Delhi, India     |
| 3 |             | Rahul Kumar | rahul.kumar@email.com | 9812345678 | Mumbai, India    |
| 4 |             | Sneha Patel | sneha.patel@email.com | 9123456789 | Ahmedabad, India |
| 5 |             | Vikas Roy   | vikas.roy@email.com   | 9001122334 | Kolkata, India   |
| 7 |             | tushar      | tushar@gmail.com      | 234567890  | banglore         |
| 8 |             | suhas       | suhas@gmail.com       | 9876543212 | banglore         |

## READ

E-Commerce Database Management System - Logged in as: admin

E-Commerce Database Management System

Select Table

Customers

Orders

Order Details

Products

Returns

Reviews

Shipments

Stored Procedures

Get Order History

Generate Sales Report

Functions

Customer LTV

Product Revenue

Create

Read/Refresh

Update

Delete

Search: 1

Search

| customer_id | name        | email                 | phone      | address          |
|-------------|-------------|-----------------------|------------|------------------|
| 1           | Amit Sharma | amit.sharma@email.com | 9876543210 | Bangalore, India |
| 2           | Pooja Singh | pooja.singh@email.com | 9876501234 | Delhi, India     |
| 3           | Rahul Kumar | rahul.kumar@email.com | 9812345678 | Mumbai, India    |
| 4           | Sneha Patel | sneha.patel@email.com | 9123456789 | Ahmedabad, India |
| 7           | tushar      | tushar@gmail.com      | 234567890  | banglore         |
| 8           | suhas       | suhas@gmail.com       | 9876543212 | banglore         |

| Search: 1 |             |             |                       |            | Search           |
|-----------|-------------|-------------|-----------------------|------------|------------------|
|           | customer_id | name        | email                 | phone      | address          |
| 1         |             | Amit Sharma | amit.sharma@email.com | 9876543210 | Bangalore, India |

## UPDATE

 Update Customers Record

### Update Customers Details

|              |                                              |                   |
|--------------|----------------------------------------------|-------------------|
| customer_id: | <input type="text" value="8"/>               | <div>Update</div> |
| name:        | <input type="text" value="suhas"/>           |                   |
| email:       | <input type="text" value="suhas@gmail.com"/> |                   |
| phone:       | <input type="text" value="9876543212"/>      |                   |
| address:     | <input type="text" value="tumkur"/>          |                   |

 Success

X

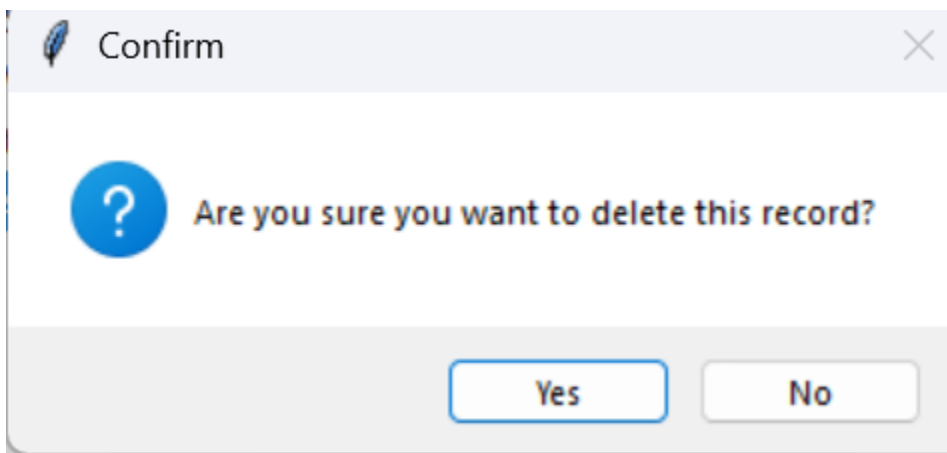


Record updated successfully!

OK

|   | customer_id | name        | email                 | phone      | address          |
|---|-------------|-------------|-----------------------|------------|------------------|
| 1 |             | Amit Sharma | amit.sharma@email.com | 9876543210 | Bangalore, India |
| 2 |             | Pooja Singh | pooja.singh@email.com | 9876501234 | Delhi, India     |
| 3 |             | Rahul Kumar | rahul.kumar@email.com | 9812345678 | Mumbai, India    |
| 4 |             | Sneha Patel | sneha.patel@email.com | 9123456789 | Ahmedabad, India |
| 5 |             | Vikas Roy   | vikas.roy@email.com   | 9001122334 | Kolkata, India   |
| 7 |             | tushar      | tushar@gmail.com      | 234567890  | banglore         |
| 8 |             | suhas       | suhas@gmail.com       | 9876543212 | tumkur           |

## DELETE



|   | customer_id | name        | email                 | phone      | address          |
|---|-------------|-------------|-----------------------|------------|------------------|
| 1 |             | Amit Sharma | amit.sharma@email.com | 9876543210 | Bangalore, India |
| 2 |             | Pooja Singh | pooja.singh@email.com | 9876501234 | Delhi, India     |
| 3 |             | Rahul Kumar | rahul.kumar@email.com | 9812345678 | Mumbai, India    |
| 4 |             | Sneha Patel | sneha.patel@email.com | 9123456789 | Ahmedabad, India |
| 5 |             | Vikas Roy   | vikas.roy@email.com   | 9001122334 | Kolkata, India   |
| 7 |             | tushar      | tushar@gmail.com      | 234567890  | banglore         |

## 9. Application Functionalities & GUI Screenshots

Functionalities:

- Login system
- Table selection
- CRUD: Create, Read, Update, Delete
- Search functionality
- Stored procedure execution
- Revenue & LTV calculation
- Sales report generation

# Functional requirements

## 1. Order Placement & Storage

- Record new orders and their line items (ORDERS, ORDER\_DETAILS) with order date, customer, status and item prices.

## 2. Automatic Inventory Update

- On inserting an order line, automatically decrement the product's stock\_quantity so inventory reflects sales. (trigger)

## 3. Prevent Negative Stock

- Block updates that would set stock\_quantity below zero and raise an error to preserve data integrity. (trigger)

## 4. Automatic Order Total Calculation

- Recalculate and update ORDERS.total\_amount when order details change so the order total always reflects line items. (trigger)

## 5. Shipment Tracking

- Store and link shipment records (tracking number, carrier info, delivery date) to orders for tracking and status queries.

## 6. Customer Order History Retrieval

- Provide a procedure to return a customer's full order history with products, quantities, prices and shipment details. (procedure)

## 7. Returns Processing & Stock Restoration

- Process return requests, update return status, and restore stock for approved returns via procedure/trigger.

## 8. Sales Reporting by Date Range

- Generate daily sales summaries (orders count, unique customers, items sold, total revenue, avg order value) for a given date range. (procedure)

## 9. Product Revenue Calculation

- Compute total revenue for a given product excluding cancelled orders via a database function.

## 10. Customer Lifetime Value (LTV)

- Compute lifetime spending for a customer across all non-cancelled orders via a function.

## 11. Average Product Rating

- Compute average rating for a product from REVIEWS to support product pages and quality analytics. (function)

## 12. Concurrency & Transaction Safety

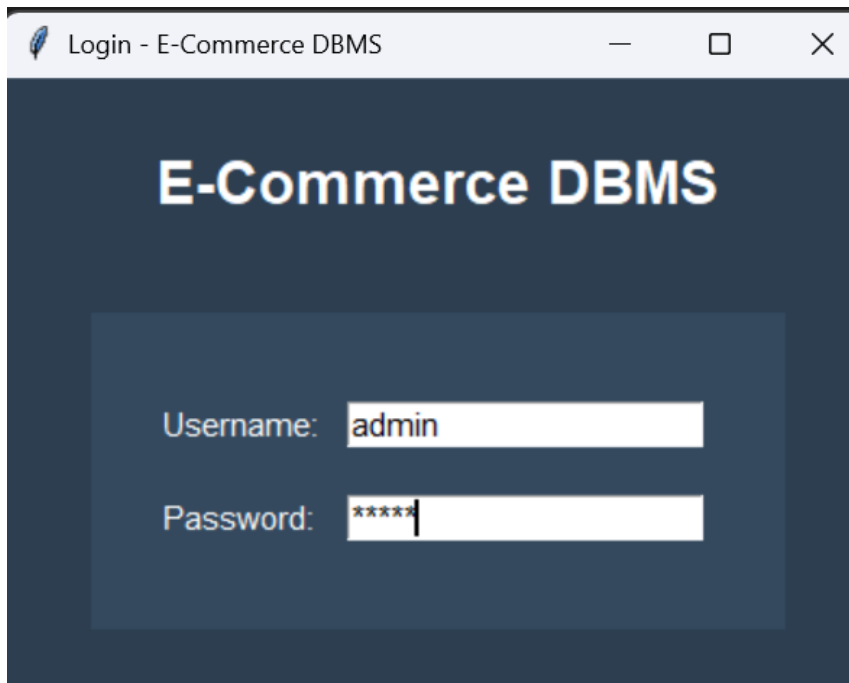
- Ensure updates that affect stock and totals run in safe transactional contexts (triggers and procedures should be used with client transactions to avoid race conditions).

## 13. Error Handling & Notifications

- Signal clear errors for invalid operations (e.g., attempting to reduce stock below zero) and return human-readable messages from procedures (e.g., return processing result).

## 14. Data Audit / Logging (optional but recommended)

- Record changes to critical objects (stock changes, return approvals) for audit and troubleshooting (can be implemented with log tables and triggers).



The image shows a web browser window titled "Login - E-Commerce DBMS". The main content area has a dark blue background with the text "E-Commerce DBMS" in large, bold, white letters. Below this, there is a light blue rectangular box containing a login form. The form has two fields: "Username:" with the text "admin" entered, and "Password:" with "\*\*\*\*\*" entered. The browser window includes standard OS controls (minimize, maximize, close) in the top right corner.

## Get Customer Order History

Customer ID:

Order History for Customer ID: 2

| product_name   | quantity | price   | order_date          | status    | tracking_number | total_amount | delivery_date       |
|----------------|----------|---------|---------------------|-----------|-----------------|--------------|---------------------|
| Wireless Mouse | 1        | 4150.00 | 2025-08-02 00:00:00 | Delivered | TRK002          | 50.00        | 2025-08-02 00:00:00 |
| 16GB Pen Drive | 2        | 700.00  | 2025-08-02 00:00:00 | Delivered | TRK002          | 150.00       | 2025-08-02 00:00:00 |
| Wireless Mouse | 2        | 1100.00 | 2025-08-02 00:00:00 | Delivered | TRK002          | 202          | 2025-08-02 00:00:00 |

Total records found: 3

Execute Procedure

## Calculate Customer LTV

Customer ID:

**Customer LTV: \$900.00**

This represents the total lifetime value for customer ID 3

Calculate LTV

 Product Revenue

—

□

×

## Calculate Product Revenue

Product ID:

## Product Revenue: \$1400.00

This represents the total revenue generated by product ID 3

Calculate Revenue

## 10. Triggers / Procedures / Functions

### 1)TRIGGERS:

- **update\_stock\_after\_order**

DELIMITER //

CREATE TRIGGER update\_stock\_after\_order

AFTER INSERT ON ORDER\_DETAILS

FOR EACH ROW

BEGIN

    UPDATE PRODUCTS

    SET stock\_quantity = stock\_quantity - NEW.quantity

    WHERE product\_id = NEW.product\_id;

END//

DELIMITER ;



- **prevent\_negative\_stock**

DELIMITER //

CREATE TRIGGER prevent\_negative\_stock

BEFORE UPDATE ON PRODUCTS

FOR EACH ROW

BEGIN

IF NEW.stock\_quantity < 0 THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE\_TEXT = 'Error: Stock quantity cannot be negative';

END IF;

END//

DELIMITER ;

- **update\_order\_total**

DELIMITER //

CREATE TRIGGER update\_order\_total

AFTER INSERT ON ORDER\_DETAILS

FOR EACH ROW

BEGIN

UPDATE ORDERS

SET total\_amount = (

SELECT SUM(quantity \* price)

FROM ORDER\_DETAILS

WHERE order\_id = NEW.order\_id

```
)  
  
    WHERE order_id = NEW.order_id;  
  
END//  
  
DELIMITER ;
```

- **log\_return\_request**

```
DELIMITER //  
  
CREATE TRIGGER log_return_request  
  
AFTER INSERT ON RETURNS  
  
FOR EACH ROW  
  
BEGIN  
  
    IF NEW.status = 'Approved' THEN  
  
        UPDATE PRODUCTS p  
  
        INNER JOIN ORDER_DETAILS od ON p.product_id = od.product_id  
  
        SET p.stock_quantity = p.stock_quantity + od.quantity  
  
        WHERE od.order_id = NEW.order_id  
  
        AND od.product_id = NEW.product_id;  
  
    END IF;  
  
END//  
  
DELIMITER ;
```

## **2)PROCEDURES:**

- **get\_customer\_order\_history**

```
DELIMITER //  
  
CREATE PROCEDURE get_customer_order_history(IN cust_id INT)  
  
BEGIN
```

```

SELECT

    o.order_id,

    o.order_date,

    o.status,

    o.total_amount,

    p.name AS product_name,

    od.quantity,

    od.price,

    s.tracking_number,

    s.delivery_date

FROM ORDERS o

INNER JOIN ORDER_DETAILS od ON o.order_id = od.order_id

INNER JOIN PRODUCTS p ON od.product_id = p.product_id

LEFT JOIN SHIPMENTS s ON o.order_id = s.order_id

WHERE o.customer_id = cust_id

ORDER BY o.order_date DESC;

END//

DELIMITER ;

```

- **process\_product\_return**

```

DELIMITER //

CREATE PROCEDURE process_product_return(

    IN p_return_id INT,

    IN p_new_status VARCHAR(20)

```

)

BEGIN

DECLARE v\_order\_id INT;

DECLARE v\_product\_id INT;

DECLARE v\_quantity INT;

SELECT order\_id, product\_id INTO v\_order\_id, v\_product\_id

FROM RETURNS WHERE return\_id = p\_return\_id;

SELECT quantity INTO v\_quantity

FROM ORDER\_DETAILS WHERE order\_id = v\_order\_id AND product\_id =  
v\_product\_id;

UPDATE RETURNS SET status = p\_new\_status WHERE return\_id = p\_return\_id;

IF p\_new\_status = 'Approved' THEN

UPDATE PRODUCTS SET stock\_quantity = stock\_quantity + v\_quantity WHERE  
product\_id = v\_product\_id;

END IF;

SELECT CONCAT('Return #', p\_return\_id, ' processed successfully. Status: ',  
p\_new\_status) AS message;

END//

DELIMITER ;

- **generate\_sales\_report**

DELIMITER //

CREATE PROCEDURE generate\_sales\_report(

    IN start\_date DATE,

    IN end\_date DATE

)

BEGIN

    SELECT

        DATE(o.order\_date) AS sale\_date,

        COUNT(DISTINCT o.order\_id) AS total\_orders,

        COUNT(DISTINCT o.customer\_id) AS unique\_customers,

        SUM(od.quantity) AS total\_items\_sold,

        SUM(od.quantity \* od.price) AS total\_revenue,

        AVG(o.total\_amount) AS avg\_order\_value

    FROM ORDERS o

    INNER JOIN ORDER\_DETAILS od ON o.order\_id = od.order\_id

    WHERE DATE(o.order\_date) BETWEEN start\_date AND end\_date

    AND o.status != 'Cancelled'

    GROUP BY DATE(o.order\_date)

    ORDER BY sale\_date DESC;

END//

DELIMITER ;

### 3)FUNCTIONS:

- **calculate\_product\_revenue**

DELIMITER //

CREATE FUNCTION calculate\_product\_revenue(prod\_id INT)

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

    DECLARE total\_revenue DECIMAL(10,2);

    SELECT COALESCE(SUM(od.quantity \* od.price), 0)

    INTO total\_revenue

    FROM ORDER\_DETAILS od

    INNER JOIN ORDERS o ON od.order\_id = o.order\_id

    WHERE od.product\_id = prod\_id

    AND o.status != 'Cancelled';

    RETURN total\_revenue;

END//

DELIMITER ;

- **calculate\_customer\_ltv**

DELIMITER //

CREATE FUNCTION calculate\_customer\_ltv(cust\_id INT)

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

    DECLARE lifetime\_value DECIMAL(10,2);

```

SELECT COALESCE(SUM(total_amount), 0) INTO lifetime_value
FROM ORDERS WHERE customer_id = cust_id AND status != 'Cancelled';

RETURN lifetime_value;

END//

DELIMITER ;

```

- **get\_average\_rating**

```

DELIMITER //

CREATE FUNCTION get_average_rating(prod_id INT)
RETURNS DECIMAL(3,2)
DETERMINISTIC
BEGIN

    DECLARE avg_rating DECIMAL(3,2);

    SELECT COALESCE(AVG(rating), 0) INTO avg_rating
    FROM REVIEWS WHERE product_id = prod_id;

    RETURN avg_rating;

END//

DELIMITER ;

```

## 4) Nested-query

### a) Total revenue per order using nested subquery

```

SELECT o.order_id,
       o.customer_id,
       (SELECT SUM(quantity * price) FROM ORDER_DETAILS od WHERE od.order_id =
o.order_id) AS calc_total

```

```
FROM ORDERS o  
WHERE o.order_id = 1;
```

**b) Find customers who bought a specific product**

```
SELECT * FROM CUSTOMERS  
WHERE customer_id IN (  
    SELECT o.customer_id FROM ORDERS o  
    WHERE o.order_id IN (  
        SELECT order_id FROM ORDER_DETAILS WHERE product_id = 2  
    )  
);
```

**5) Join examples**

**a) Inner join — order lines with product names**

```
SELECT od.order_id, p.product_id, p.name, od.quantity, od.price  
FROM ORDER_DETAILS od  
JOIN PRODUCTS p ON od.product_id = p.product_id;
```

**b) Left join — orders with shipment info (even if no shipment yet)**

```
SELECT o.order_id, o.order_date, s.tracking_number, s.delivery_date, s.status  
FROM ORDERS o  
LEFT JOIN SHIPMENTS s ON o.order_id = s.order_id;
```



**c) Multi-table join — customer order history (same as procedure)**

```
SELECT c.name, o.order_id, p.name AS product, od.quantity, od.price,  
s.tracking_number  
  
FROM CUSTOMERS c  
  
JOIN ORDERS o ON c.customer_id = o.customer_id  
  
JOIN ORDER_DETAILS od ON o.order_id = od.order_id  
  
JOIN PRODUCTS p ON od.product_id = p.product_id  
  
LEFT JOIN SHIPMENTS s ON o.order_id = s.order_id  
  
WHERE c.customer_id = 1;
```

**6) Aggregate queries**

**a) Total items sold per product**

```
SELECT od.product_id, p.name, SUM(od.quantity) AS total_sold  
  
FROM ORDER_DETAILS od  
  
JOIN PRODUCTS p ON od.product_id = p.product_id  
  
GROUP BY od.product_id, p.name  
  
ORDER BY total_sold DESC;
```

**b) Daily sales summary**

```
SELECT DATE(o.order_date) AS sale_date,  
        COUNT(DISTINCT o.order_id) AS total_orders,  
        SUM(od.quantity) AS total_items_sold,  
        SUM(od.quantity * od.price) AS total_revenue  
  
FROM ORDERS o
```

```
JOIN ORDER_DETAILS od ON o.order_id = od.order_id
```

```
WHERE o.status != 'Cancelled'
```

```
GROUP BY DATE(o.order_date)
```

```
ORDER BY sale_date DESC;
```

### **c) Average rating per product**

```
SELECT p.product_id, p.name, COALESCE(AVG(r.rating),0) AS avg_rating,  
COUNT(r.review_id) AS review_count
```

```
FROM PRODUCTS p
```

```
LEFT JOIN REVIEWS r ON p.product_id = r.product_id
```

```
GROUP BY p.product_id, p.name
```

```
ORDER BY avg_rating DESC;
```

## **7) Example nested + aggregate**

**Find products where average rating  $\geq 4$  and revenue > 1000:**

```
SELECT p.product_id, p.name,
```

```
    (SELECT COALESCE(AVG(rating),0) FROM REVIEWS r WHERE r.product_id =  
p.product_id) AS avg_rating,
```

```
    calculate_product_revenue(p.product_id) AS revenue
```

```
FROM PRODUCTS p
```

```
HAVING avg_rating >= 4 AND revenue > 1000;
```

## 11. Code Snippets (Summarized)

### A. Invoking Stored Procedures (from Python GUI)

#### 1) Calling `get_customer_order_history`

```
cursor = self.connection.cursor()

cursor.callproc('get_customer_order_history', [customer_id])
```

```
for result in cursor.stored_results():
```

```
    rows = result.fetchall()
```

```
    for row in rows:
```

```
        print(row)
```

```
cursor.close()
```

#### 2) Calling `generate_sales_report`

```
cursor = self.connection.cursor()
```

```
cursor.callproc('generate_sales_report', [start_date, end_date])
```

```
for result in cursor.stored_results():
```

```
    data = result.fetchall()
```

```
    for row in data:
```

```
        print(row)
```

```
cursor.close()
```

#### 3) Calling `process_product_return`

```
cursor = self.connection.cursor()
```

```
cursor.callproc('process_product_return', [return_id, new_status])
```

```
self.connection.commit()
```

```
for result in cursor.stored_results():
```

```
    print(result.fetchone())
```

```
cursor.close()
```

## **B. Invoking SQL Functions from Python**

### **1) Calling calculate\_customer\_ltv**

```
cursor = self.connection.cursor()
```

```
query = f"SELECT calculate_customer_ltv({customer_id})"
```

```
cursor.execute(query)
```

```
ltv = cursor.fetchone()[0]
```

```
cursor.close()
```

### **2) Calling calculate\_product\_revenue**

```
cursor = self.connection.cursor()
```

```
query = f"SELECT calculate_product_revenue({product_id})"
```

```
cursor.execute(query)
```

```
revenue = cursor.fetchone()[0]
```

```
cursor.close()
```

### **3) Calling get\_average\_rating**

```
cursor = self.connection.cursor()
```

```
cursor.execute(f"SELECT get_average_rating({product_id})")
```

```
rating = cursor.fetchone()[0]
```

```
cursor.close()
```

## C. Trigger Execution

### 1) Trigger: update\_stock\_after\_order

This trigger executes when a new order item is inserted.

```
cursor.execute(
    "INSERT INTO ORDER_DETAILS(order_detail_id, order_id, product_id, quantity, price) "
    "VALUES (%s, %s, %s, %s, %s)",
    (od_id, order_id, product_id, quantity, price)
)
self.connection.commit()
```

### 2) Trigger: update\_order\_total

Same insert as above automatically fires total recalculation:

AFTER INSERT ON ORDER\_DETAILS

### 3) Trigger: prevent\_negative\_stock

Triggered when updating product stock from GUI:

```
cursor.execute(
    "UPDATE PRODUCTS SET stock_quantity = %s WHERE product_id = %s",
    (new_stock, product_id)
)
self.connection.commit()
```

### 4) Trigger: log\_return\_request

Activated when inserting a return entry with status 'Approved':

```
cursor.execute(
    "INSERT INTO RETURNS(return_id, order_id, product_id, return_date, reason, status) "
    "VALUES (%s, %s, %s, %s, %s, %s)",
```

```
(ret_id, order_id, product_id, date, reason, "Approved")
)

self.connection.commit()
```

## 12. SQL QUERIES

-- Create the CUSTOMERS table

```
CREATE TABLE CUSTOMERS (

    customer_id INT PRIMARY KEY,

    name VARCHAR(100) NOT NULL,

    email VARCHAR(100) NOT NULL UNIQUE,

    phone VARCHAR(20) NOT NULL,

    address VARCHAR(255) NOT NULL

);
```

-- Create the PRODUCTS table

```
CREATE TABLE PRODUCTS (

    product_id INT PRIMARY KEY,

    name VARCHAR(100) NOT NULL,

    description TEXT,

    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),

    stock_quantity INT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0));
```

-- Create the ORDERS table

```
CREATE TABLE ORDERS (  
    order_id INT PRIMARY KEY,  
    customer_id INT NOT NULL,  
    order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) NOT NULL DEFAULT 'Pending',  
    total_amount DECIMAL(10,2) NOT NULL CHECK (total_amount >= 0),  
    CONSTRAINT fk_orders_customers FOREIGN KEY (customer_id)  
        REFERENCES CUSTOMERS (customer_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

-- Create the ORDER\_DETAILS table

```
CREATE TABLE ORDER_DETAILS (  
    order_detail_id INT PRIMARY KEY,  
    order_id INT NOT NULL,  
    product_id INT NULL,  
    quantity INT NOT NULL CHECK (quantity > 0),  
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
    CONSTRAINT fk_orderdetails_orders FOREIGN KEY (order_id)  
        REFERENCES ORDERS (order_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE,
```

```
CONSTRAINT fk_orderdetails_products FOREIGN KEY (product_id)
REFERENCES PRODUCTS(product_id)
ON DELETE SET NULL
ON UPDATE CASCADE
);
```

-- Create the SHIPMENTS table

```
CREATE TABLE SHIPMENTS (
shipment_id INT PRIMARY KEY,
order_id INT NOT NULL,
shipment_date DATE NOT NULL,
delivery_date DATE,
status VARCHAR(20) NOT NULL DEFAULT 'Shipped',
tracking_number VARCHAR(50) UNIQUE,
CONSTRAINT fk_shipments_orders FOREIGN KEY (order_id)
REFERENCES ORDERS(order_id)
ON DELETE CASCADE
ON UPDATE CASCADE
);
```

-- Create the RETURNS table

```
CREATE TABLE RETURNS (
return_id INT PRIMARY KEY,
order_id INT NULL,
```



```
product_id INT NULL,  
return_date DATE NOT NULL,  
reason VARCHAR(255) NOT NULL,  
status VARCHAR(20) NOT NULL DEFAULT 'Processing',  
CONSTRAINT fk_returns_orders FOREIGN KEY (order_id)  
    REFERENCES ORDERS(order_id)  
    ON DELETE CASCADE  
    ON UPDATE SET NULL,  
CONSTRAINT fk_returns_products FOREIGN KEY (product_id)  
    REFERENCES PRODUCTS(product_id)  
    ON DELETE SET NULL  
    ON UPDATE SET NULL  
);
```

-- Create the REVIEWS table

```
CREATE TABLE REVIEWS (  
    review_id INT PRIMARY KEY,  
    product_id INT NULL,  
    customer_id INT NOT NULL,  
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),  
    comment TEXT,  
    review_date DATE NOT NULL,  
    CONSTRAINT fk_reviews_products FOREIGN KEY (product_id)  
        REFERENCES PRODUCTS(product_id)
```

```
ON DELETE CASCADE
ON UPDATE CASCADE,
CONSTRAINT fk_reviews_customers FOREIGN KEY (customer_id)
REFERENCES CUSTOMERS(customer_id)
ON DELETE CASCADE
ON UPDATE CASCADE
);

-- CUSTOMERS
INSERT INTO CUSTOMERS VALUES
(1, 'Amit Sharma', 'amit.sharma@email.com', '9876543210', 'Bangalore, India'),
(2, 'Pooja Singh', 'pooja.singh@email.com', '9876501234', 'Delhi, India'),
(3, 'Rahul Kumar', 'rahul.kumar@email.com', '9812345678', 'Mumbai, India'),
(4, 'Sneha Patel', 'sneha.patel@email.com', '9123456789', 'Ahmedabad, India'),
(5, 'Vikas Roy', 'vikas.roy@email.com', '9001122334', 'Kolkata, India');

-- PRODUCTS (Assuming category_id as 1 for all)
INSERT INTO PRODUCTS VALUES
(1, 'Wireless Mouse', 'Ergonomic Wireless Mouse', 550.00, 80),
(2, 'Mechanical Keyboard', 'RGB Backlit Mechanical Keyboard', 2500.00, 50),
(3, '16GB Pen Drive', 'USB 3.1 High Speed Pen Drive', 700.00, 120),
(4, 'Laptop Stand', 'Adjustable Laptop Stand', 900.00, 40),
(5, 'Bluetooth Speaker', 'Portable Bluetooth Speaker', 1200.00, 60);
```

-- ORDERS

INSERT INTO ORDERS VALUES

(1, 1, '2025-08-01', 'Shipped', 3200.00),  
(2, 2, '2025-08-02', 'Delivered', 1450.00),  
(3, 3, '2025-08-03', 'Processing', 900.00),  
(4, 4, '2025-08-04', 'Cancelled', 2500.00),  
(5, 5, '2025-08-05', 'Shipped', 1950.00);

-- ORDER\_DETAILS

INSERT INTO ORDER\_DETAILS VALUES

(1, 1, 2, 1, 2500.00),  
(2, 1, 5, 1, 1200.00),  
(3, 2, 1, 1, 550.00),  
(4, 2, 3, 2, 700.00),  
(5, 3, 4, 1, 900.00);

-- SHIPMENTS

INSERT INTO SHIPMENTS VALUES

(1, 1, '2025-08-02', '2025-08-05', 'Delivered', 'TRK001'),  
(2, 2, '2025-08-03', '2025-08-06', 'Delivered', 'TRK002'),  
(3, 3, '2025-08-04', NULL, 'Shipped', 'TRK003'),  
(4, 4, '2025-08-06', NULL, 'Cancelled', 'TRK004'),  
(5, 5, '2025-08-07', NULL, 'Shipped', 'TRK005');

-- RETURNS

INSERT INTO RETURNS VALUES

(1, 2, 3, '2025-08-09', 'Defective item', 'Processed'),  
(2, 1, 5, '2025-08-10', 'Not as described', 'Processing'),  
(3, 1, 2, '2025-08-10', 'Wrong item sent', 'Processing'),  
(4, 2, 1, '2025-08-11', 'Damaged in transit', 'Processed'),  
(5, 3, 4, '2025-08-12', 'Changed mind', 'Pending');

-- REVIEWS

INSERT INTO REVIEWS VALUES

(1, 1, 1, 4, 'Works well, satisfied!', '2025-08-06'),  
(2, 3, 2, 5, 'Very fast data transfer.', '2025-08-07'),  
(3, 5, 1, 2, 'Poor sound quality.', '2025-08-08'),  
(4, 4, 5, 5, 'Sturdy and versatile stand!', '2025-08-09'),  
(5, 2, 3, 3, 'Good keyboard, average keys.', '2025-08-10');

DELIMITER //

CREATE TRIGGER update\_stock\_after\_order

AFTER INSERT ON ORDER\_DETAILS

FOR EACH ROW

BEGIN

    UPDATE PRODUCTS

    SET stock\_quantity = stock\_quantity - NEW.quantity

```
WHERE product_id = NEW.product_id;

END//

DELIMITER ;


DELIMITER //

CREATE TRIGGER prevent_negative_stock

BEFORE UPDATE ON PRODUCTS

FOR EACH ROW

BEGIN

    IF NEW.stock_quantity < 0 THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Error: Stock quantity cannot be negative';

    END IF;

END//

DELIMITER ;


DELIMITER //

CREATE TRIGGER update_order_total

AFTER INSERT ON ORDER_DETAILS

FOR EACH ROW

BEGIN

    UPDATE ORDERS

    SET total_amount = (

        SELECT SUM(quantity * price)
```

```

        FROM ORDER_DETAILS

        WHERE order_id = NEW.order_id

    )

    WHERE order_id = NEW.order_id;

END//

DELIMITER ;


DELIMITER //

CREATE TRIGGER log_return_request

AFTER INSERT ON RETURNS

FOR EACH ROW

BEGIN

    IF NEW.status = 'Approved' THEN

        UPDATE PRODUCTS p

        INNER JOIN ORDER_DETAILS od ON p.product_id = od.product_id

        SET p.stock_quantity = p.stock_quantity + od.quantity

        WHERE od.order_id = NEW.order_id

        AND od.product_id = NEW.product_id;

    END IF;

END//

DELIMITER ;


DELIMITER //

CREATE PROCEDURE get_customer_order_history(IN cust_id INT)

```

```
BEGIN

SELECT

    o.order_id,

    o.order_date,

    o.status,

    o.total_amount,

    p.name AS product_name,

    od.quantity,

    od.price,

    s.tracking_number,

    s.delivery_date

FROM ORDERS o

INNER JOIN ORDER_DETAILS od ON o.order_id = od.order_id

INNER JOIN PRODUCTS p ON od.product_id = p.product_id

LEFT JOIN SHIPMENTS s ON o.order_id = s.order_id

WHERE o.customer_id = cust_id

ORDER BY o.order_date DESC;

END//

DELIMITER ;


DELIMITER //

CREATE PROCEDURE process_product_return(

    IN p_return_id INT,

    IN p_new_status VARCHAR(20)
```

)

BEGIN

DECLARE v\_order\_id INT;

DECLARE v\_product\_id INT;

DECLARE v\_quantity INT;

-- Get return details

SELECT order\_id, product\_id INTO v\_order\_id, v\_product\_id

FROM RETURNS WHERE return\_id = p\_return\_id;

-- Get quantity from order details

SELECT quantity INTO v\_quantity

FROM ORDER\_DETAILS WHERE order\_id = v\_order\_id AND product\_id =  
v\_product\_id;

-- Update return status

UPDATE RETURNS SET status = p\_new\_status WHERE return\_id = p\_return\_id;

-- If approved, restore stock

IF p\_new\_status = 'Approved' THEN

UPDATE PRODUCTS SET stock\_quantity = stock\_quantity + v\_quantity WHERE  
product\_id = v\_product\_id;

END IF;



```
SELECT CONCAT('Return #', p_return_id, ' processed successfully. Status: ',  
p_new_status) AS message;
```

```
END//
```

```
DELIMITER ;
```

```
DELIMITER //
```

```
CREATE PROCEDURE generate_sales_report(  
    IN start_date DATE,  
    IN end_date DATE  
)  
BEGIN
```

```
    SELECT
```

```
        DATE(o.order_date) AS sale_date,  
        COUNT(DISTINCT o.order_id) AS total_orders,  
        COUNT(DISTINCT o.customer_id) AS unique_customers,  
        SUM(od.quantity) AS total_items_sold,  
        SUM(od.quantity * od.price) AS total_revenue,  
        AVG(o.total_amount) AS avg_order_value  
    FROM ORDERS o  
    INNER JOIN ORDER_DETAILS od ON o.order_id = od.order_id  
    WHERE DATE(o.order_date) BETWEEN start_date AND end_date  
    AND o.status != 'Cancelled'  
    GROUP BY DATE(o.order_date)  
    ORDER BY sale_date DESC;
```

```
END//

DELIMITER ;


DELIMITER //

CREATE FUNCTION calculate_product_revenue(prod_id INT)

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

    DECLARE total_revenue DECIMAL(10,2);

    SELECT COALESCE(SUM(od.quantity * od.price), 0)

    INTO total_revenue

    FROM ORDER_DETAILS od

    INNER JOIN ORDERS o ON od.order_id = o.order_id

    WHERE od.product_id = prod_id

    AND o.status != 'Cancelled';

    RETURN total_revenue;

END//

DELIMITER ;


DELIMITER //

CREATE FUNCTION calculate_customer_ltv(cust_id INT)

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN
```

```

DECLARE lifetime_value DECIMAL(10,2);

SELECT COALESCE(SUM(total_amount), 0) INTO lifetime_value

FROM ORDERS WHERE customer_id = cust_id AND status != 'Cancelled';

RETURN lifetime_value;

END//

DELIMITER ;


DELIMITER //

CREATE FUNCTION get_average_rating(prod_id INT)

RETURNS DECIMAL(3,2)

DETERMINISTIC

BEGIN

    DECLARE avg_rating DECIMAL(3,2);

    SELECT COALESCE(AVG(rating), 0) INTO avg_rating

    FROM REVIEWS WHERE product_id = prod_id;

    RETURN avg_rating;

END//

DELIMITER ;

```

### 13. GitHub Repository Link

<https://github.com/Vikramhalli/DBMS--E--Commerce-Order-Tracking-System->